AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 6.5b: KFE, Deep MFG, and Lab Preview

Zhigang Feng

Spring 2026

Topic 6.5b Agenda

1. **Mean-Field Games (continued)** — KFE, discretisation, master equation, deep MFG
2. **Lab Preview and Wrap-up**

Arc: T1 Aiyagari + computation → T2a Why ML / Euler → T2b Global DNN → T3 KS / DeepHAM → T4 DEQN / OLG → T5a MFG HJB → **T5b MFG KFE + Lab**

Mean-Field Games (continued)

The Kolmogorov forward (Fokker–Planck) equation

For the time-dependent density $g_i(a,t)$:

$$\partial_t g_i(a,t) \;=\; -\partial_a\big(s_i(a,t)\, g_i(a,t)\big) - \lambda_i\, g_i(a,t) + \lambda_{-i}\, g_{-i}(a,t).$$

- First term: *drift* — mass moves at speed $s_i(a,t)$ along the a-axis.
- Second term: *Poisson outflow* from state i at rate λ_i.
- Third term: *Poisson inflow* from the other state at rate λ_{-i}.
- Boundary: at the borrowing limit $a = \underline{a}$, mass cannot leak below; we impose a reflecting boundary.
- Well-posed under standard conditions (Achdou et al. 2022).

Reading the KFE: economic intuition

- **Drift term**: the cross-section flows to the right when households are saving ($s_i > 0$) and to the left when they are dissaving.
- **Jump terms**: at every point in a, mass leaks from state i at rate λ_i and arrives from $-i$ at rate λ_{-i}. The two states "trade" agents.
- **Boundary at $\underline{a}$**: reflecting boundary — mass that hits the borrowing constraint cannot continue leftward; instead it accumulates at $\underline{a}$ for as long as $s_i(\underline{a}) \leq 0$.
- **Total mass conserved**: $\int g_1 + \int g_2 = 1$ at all times.
- Analogy: water flowing through two interconnected tanks with leaks at known rates.

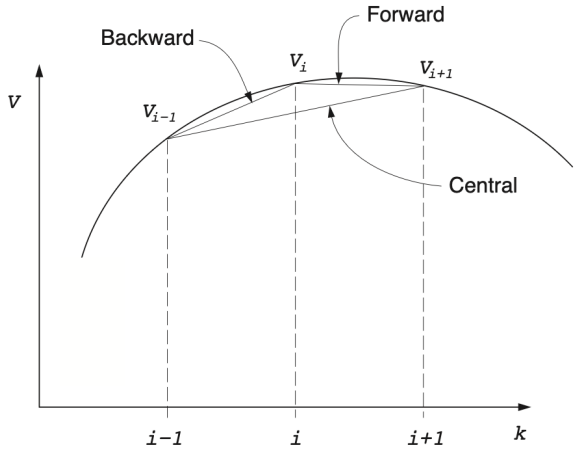
Stationary KFE

At the stationary equilibrium ($\partial_t = 0$):

$$0 \;=\; -\partial_a\big(s_i(a)\, g_i(a)\big) - \lambda_i\, g_i(a) + \lambda_{-i}\, g_{-i}(a), \quad i = 1, 2.$$

- Two coupled ODEs in g_1, g_2, plus the normalisation $\int g_1 + \int g_2 = 1$.
- Reflecting boundary at $\underline{a}$, integrability at $a \to \infty$.
- Standard discretisation gives a sparse linear system $A^\top g = 0$ where A is the discretised HJB infinitesimal generator (next slide).

Discretisation and the upwind scheme



- Discretise a on a grid $\{a_1, \ldots, a_N\}$. The HJB needs an approximation of $v_i'(a)$.
- **Upwind scheme**: pick the forward or backward difference depending on the *sign of the optimal drift*:

HJB and KFE: A vs. $A^\top$ duality

After upwind discretisation, the HJB takes the form

$$\rho\,\mathbf{v} \;=\; \mathbf{u}(\mathbf{c}) + A(\mathbf{v})\,\mathbf{v},$$

where $A(\mathbf{v}) \in \mathbb{R}^{2N\times 2N}$ is the discretised infinitesimal generator (sparse, tridiagonal-plus-jumps).

The discretised stationary KFE is

$$\mathbf{0} \;=\; A(\mathbf{v})^\top \mathbf{g}.$$

- The *same* matrix A generates both equations: HJB on the right, KFE on the left (transpose).
- This duality is at the heart of the Achdou et al. approach: solving HJB gives you the KFE almost for free.

Putting it all together: outer fixed point on r

1. Guess r.
2. Solve HJB by iterating
$$\rho \mathbf{v} - A(\mathbf{v})\mathbf{v} \;=\; \mathbf{u}(\mathbf{c})$$
to find $\mathbf{v}^*, \mathbf{c}^*, A^*$.
3. Solve KFE: find $\mathbf{g}^*$ in the null space of $A^{*\top}$, normalised to $\mathbf{1}^\top \mathbf{g}^* = 1$.
4. Aggregate: $K^* = \sum_j a_j \big(g^*_{1,j} + g^*_{2,j}\big)$.
5. Update r from the firm FOC at K^* and iterate until convergence.

This is the Achdou–Lasry–Lions–Moll algorithm for continuous-time HA models. It is fast, accurate, and (with deep learning) extends to high-dimensional state spaces.

Transition dynamics: forward–backward system (advanced — skip on first pass)

For non-stationary problems (e.g. MIT shocks, business cycles), HJB and KFE become time-dependent and the system is *forward–backward*:

$$\begin{cases} -\partial_t v_i + \rho v_i \;=\; \max_c \{\cdots\}, & \text{(backward in time)} \\ \partial_t g_i \;=\; -\partial_a(s_i g_i) - \lambda_i g_i + \lambda_{-i} g_{-i}. & \text{(forward in time)} \end{cases}$$

- Boundary in time: $v_i(\cdot, T) =$ terminal condition (often the stationary v_i^*); $g_i(\cdot, 0) =$ initial condition.
- Numerical scheme: *shooting* or *Newton's method* on the residuals — iterate backward HJB given g, then forward KFE given v, until self-consistent.

Master equation: distribution as state (advanced — skip on first pass)

- The master equation is a single equation that encodes both HJB and KFE simultaneously, with the *distribution itself* as a state variable.
- Schematically:

$$\rho U(x,m) \;=\; H\big(x, \nabla_x U(x,m), m\big) + \int \nabla_m U(x,m)(y)\, \mathcal{L}^* m(y)\, dy,$$

  where U is the value function defined on the product space (individual state, distribution) and $\mathcal{L}^*$ is the dual generator on the distribution.
- This is an infinite-dimensional PDE on the Wasserstein space of probability measures.
- Practical use: linearisation around the stationary equilibrium gives a tractable system for solving HA models with aggregate shocks (Bilal 2023; Alvarez–Lippi 2022).

Deep learning for MFG

- Classical numerical schemes (upwind, finite differences) suffer in high-dimensional state spaces.
- **Deep neural networks for MFG**:
  - Parameterise $v(x, m)$ and $m(x)$ by NNs.
  - Train by minimising the residual of the HJB and the KFE jointly.
  - Use Monte Carlo sampling instead of grids.
- Recent progress (Carmona–Laurière 2021, Laurière et al. 2022) shows that deep MFG solvers can handle 10–20 dimensional state spaces that are completely out of reach for grid methods.
- These methods unify the discrete-time RL approaches (DeepHAM, DEQN) with the continuous-time PDE perspective.

MFG $\leftrightarrow$ HA models: the link

- Discrete-time Aiyagari is the natural *discretisation* of continuous-time MFG with idiosyncratic Poisson income.
- Krusell–Smith with aggregate uncertainty corresponds to MFGs with a *common noise* (the aggregate shock).
- In both formulations:
  - Optimal individual behaviour depends on the cross-sectional distribution.
  - The cross-sectional distribution evolves under optimal behaviour.
  - Equilibrium is a fixed point of this circular dependence.
- MFG provides the cleanest mathematical formalism, while RL/DL provide the computational engine.
- The two approaches are now converging: deep MFG solvers borrow heavily from RL; modern deep RL borrows from continuous-time control theory.

Lab Preview and Wrap-up

Lab06 preview: Krusell–Smith via Deep Equilibrium Nets

- In the 1-hour lab you will train a Deep Equilibrium Net to solve the Krusell–Smith (1998) economy in PyTorch.
- Notebook: `Lab06_KS_HA.ipynb` (in this folder).
- By the end of the lab you will:
  - Initialise the model from a steady state computed via the sequence-Jacobian (SSJ) toolbox.
  - Build a PyTorch policy network for the savings rate.
  - Run the cloud-of-economies training loop.
  - Visualise the trained policy and the loss curves.
  - Diagnose convergence and accuracy.
- Hardware: a single GPU (Colab T4 / V100) is comfortable; CPU also works for short runs.
- *Forward pointer:* the cloud-simulation pattern in this lab (N parallel economies, batched Euler-residual minimisation) reappears in Lec07 as cloud-of-prompts batch processing in LLM pretraining — same hardware-utilisation argument, different objective.

Lab06 step-by-step

Part 1. Initialise via sequence-Jacobian:

- Solve the steady state to get asset and employment grids, transition matrices, K_{ss}, μ_{ss}.

Part 2. PyTorch implementation:

- Define the policy network (3 hidden layers, Mish, sigmoid output).
- Encode the budget constraint inside the network.
- Wrap the model physics (transition dynamics) as a class.

Part 3. The training loop (cloud simulation):

- Simulate N parallel economies.
- Compute the Euler-residual loss.
- Backprop and Adam update.
- Transport the distribution forward (no grad).

Part 4. Analysis:

- Plot the loss history (log scale).

Lab06: expected-output checkpoint

After Part 3 (training) and before Part 4 (analysis), check the lab is healthy:

- **Training loss curve.** Monotone decreasing on log scale; Euler residual $\lesssim 10^{-4}$ on a held-out validation sample.
- **Learned savings policy.** $\sigma_\theta(a, e; Z, K)$ monotone in a; employed agents save more than unemployed at the same a (precautionary motive).
- **Aggregate consistency.** K_n across cloud copies stays within $\pm 5\%$ of the SSJ steady-state K_{ss}; fan plot narrows toward steady state, not diverging.

If any of these look wrong, the trainer has not converged — inspect learning rate, batch size, and cloud size N before reading the policy plots.

Optional follow-up: Lab12_OLG.ipynb

- Once you finish the KS lab, you may want to explore `Notebooks/Lab12_OLG.ipynb`.
- This notebook implements OLG with aggregate uncertainty using JAX and stabilising homotopies (Brumm–Kubler–Scheidegger 2023).
- Highlights:
  - Two-stage homotopy: first solve a capital-only model, then introduce bonds.
  - JAX `jit` compilation for fast training-simulation episodes.
  - Detailed pedagogical commentary inside the notebook.
- Useful preparation for research projects involving life-cycle, fiscal policy, or asset-pricing applications.

Suggested readings

Aiyagari family. Bewley (1986); Huggett (1993, *JEDC*); Aiyagari (1994, *QJE*); Stokey, Lucas & Prescott (1989, Harvard UP); Carroll (2006, *Econ. Letters*).

Aggregate uncertainty. Krusell & Smith (1998, *JPE*); Kaplan, Moll & Violante (2018, *AER*).

Deep-learning HA. Han, Yang & E (2021 preprint, arXiv:2112.14377; *Quantitative Economics* 17:297–341, 2026); Azinovic, Gaegauf & Scheidegger (2022, *IER*); Brumm, Kubler & Scheidegger (2023, working paper).

Continuous-time + MFG. Achdou, Han, Lasry, Lions & Moll (2022, *REStud*); Lasry & Lions (2007, *Japanese J. Math.*); Huang, Malhamé & Caines (2006, *Communications in Information and Systems* 6(3):221–252).

Bridge: from HA models to LLMs (Lec07)

- *Same building blocks:* neural function approximation (T2a `ValueNet`, T3 DeepHAM, T4 DEQN) reappears as transformer blocks parameterising $\log p(\text{token} \mid \text{context})$.
- *Same training discipline:* sampling-based losses (Euler residuals, fictitious play) become next-token cross-entropy on cloud-of-prompts batches.
- *Same actor–critic structure:* Lec05 T3 policy gradient comes back as RLHF (Lec07 T3) — policy = LLM, reward = preference model trained on human comparisons.
- *Same scaling story:* cloud-simulation of N economies in the Lab06 DEQN trainer is the same parallelism that batches N documents in LLM pretraining.

Lec07 changes the *object* (text rather than wealth distributions) but reuses every algorithmic primitive from Lec05 and Lec06.

Recap of Lecture 6

- **Aiyagari–Bewley–Huggett**: incomplete-markets HA model and stationary equilibrium; existence/uniqueness via Feller, mixing, monotonicity.
- **Classical computation**: nested fixed-point on r; VFI on the household, eigenvector for the distribution, bisection on the price.
- **Why ML?** Curse of dimensionality once we add aggregate shocks, multiple assets, or life-cycle structure.
- **Deep dynamic programming**: policy/value networks, Euler residual losses, actor–critic.
- **DeepHAM and DEQN**: two leading deep-learning solutions for KS.
- **OLG with aggregate uncertainty**: homotopy-stabilised JAX training.
- **Mean-Field Games**: continuous-time formulation, HJB+KFE, deep MFG solvers as the unifying frontier.
- **Next lecture**: Large Language Models — a different but equally fast-moving deep-learning frontier.